

Министерство науки и высшего образования Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа радиотехники и компьютерных технологий
Кафедра системного программирования ИСП РАН
Лаборатория компиляторных технологий

Выпускная квалификационная работа магистра

Извлечение и анализ конечных автоматов из описаний аппаратуры на языке SystemVerilog

Автор:

Студент М01-405 группы
Вязовцев Андрей Викторович

Научный руководитель:

к. ф.-м. н.
Игнатъев Валерий Николаевич

Научный консультант:

почти кандидат
Чуркинян Ian



Москва 2026

Аннотация

Извлечение и анализ конечных автоматов из описаний аппаратуры на языке SystemVerilog

Вязовцев Андрей Викторович

Конечные автоматы (англ. Finite State Machines, FSM) являются широко используемым шаблоном проектирования в языках описания аппаратуры (англ. Hardware Description Languages, HDL) для моделирования управляющей логики. Несмотря на свою простоту, реализации FSM подвержены различным классам ошибок проектирования, которые трудно обнаружить с помощью традиционных методов верификации на основе моделирования. В данной работе представлен SVAN-FSM — инструмент для автоматического извлечения и статического анализа конечных автоматов из проектов на SystemVerilog. Инструмент работает на уровне списка логических элементов, строит графовые представления и выявляет недостижимые состояния, «мертвые» переходы, блокировки (deadlocks) и «живые» блокировки (livelocks). Также он обнаруживает взаимодействующие FSM для выявления межмодульных зависимостей управления и поддерживает параметризованные проекты, анализируя каждую развернутую инстанцию независимо. Результаты анализа сопоставляются с исходным кодом для повышения интерпретируемости. Инструмент был протестирован на нескольких открытых проектах RISC-V и системах, критичных с точки зрения безопасности, продемонстрировав свою эффективность на проектах промышленного масштаба.

Abstract

Finite state machines (FSMs) are a widely used design pattern in hardware description languages (HDLs) for modeling control logic. Despite their simplicity, FSM implementations are prone to various classes of design errors that are difficult to detect using conventional simulation-based verification techniques. This paper presents SVAN-FSM, a tool for automatic extraction and static analysis of FSMs from SystemVerilog designs. The tool operates on RTL netlists, builds graph-based representations, and detects unreachable states, dead transitions, deadlocks, and livelocks. It also identifies interacting FSMs to expose cross-module control dependencies and handles parameterized designs by analyzing each elaborated instance independently. Analysis results are mapped back to the source code to improve interpretability. The tool has been evaluated on multiple open-source RISC-V designs and security-critical projects, demonstrating its effectiveness on industrial-scale designs.

Содержание

1	Введение	4
2	Постановка задачи	6
3	Обзор существующих решений	7
3.1	Извлечение конечных автоматов на основе АСД-представления	7
3.2	Извлечение конечных автоматов на списке логических элементов	8
3.3	Методы анализа извлечённых моделей конечных автоматов	9
3.4	Сравнение существующих инструментов анализа	10
4	Исследование и построение решения задачи	11
4.1	Конечные автоматы и возможные ошибки в них	11
4.1.1	Основные определения	11
4.1.2	Конечные автоматы в языках описания аппаратуры	12
4.1.3	Классификация типичных ошибок в конечных автоматах	13
4.1.4	Сильно связанные компоненты и их роль в анализе	14
4.2	Инструмент SVAN и библиотека slang	15
4.3	Синтезатор yosys и его подход к извлечению конечных автоматов	16
4.3.1	Общая информация о Yosys	16
4.3.2	Представление логики переходов в виде мультиплексорных деревьев	17
4.3.3	Извлечение конечных автоматов в Yosys	18
4.3.4	Ограничения представления	19
4.4	Библиотека yosys-slang	19
5	Описание практической части	21
5.1	Общая схема работы анализатора	21
5.2	Извлечение конечных автоматов из списка логических элементов	21
5.2.1	Разбиение проводов на части	22
5.2.2	Поддержка автоматов Мили	23
5.2.3	Обработка синхронного сигнала сброса	23
5.3	Графовый анализ конечного автомата	24
5.3.1	Обнаружение мёртвых переходов	25
5.3.2	Анализ недостижимых состояний	26
5.3.3	Обнаружение дедлоков и лайвлоков	26
5.4	Сопоставление результатов анализа с исходным кодом	27
5.4.1	Отображение входов конечного автомата на выражения АСД	27
5.4.2	Восстановление имён состояний	28
5.5	Обнаружение взаимодействующих автоматов	29
6	Тестирование и результаты	31
6.1	Набор тестовых проектов	31
6.2	Результаты обнаружения	31
6.3	Классификация найденных проблем	32
6.4	Производительность	33
6.5	Обсуждение проекта OpenC910	34
6.6	Сравнение с коммерческим инструментом	34
6.7	Обработка параметризованных модулей	36
7	Заключение	37

1 Введение

Разработка цифровой интегральной схемы — это многоэтапный процесс, на каждой стадии которого требуется верификация, то есть проверка соответствия полученного описания техническому заданию или результату предыдущего этапа. Одним из ранних и наиболее важных этапов является построение модели схемы на уровне регистровых передач (англ. Register Transfer Level, RTL), которая описывает поведение схемы в терминах элементов памяти (регистров) и передачи данных между ними. Для описания RTL-моделей используются языки описания аппаратуры (англ. Hardware Description Languages, HDL), среди которых наибольшее распространение получили Verilog и его расширение SystemVerilog [1].

Одним из самых распространённых шаблонов проектирования в HDL является конечный автомат (англ. Finite State Machine, FSM). Конечный автомат позволяет сохранять текущее состояние исполнения в специальном регистре и обновлять его на последующих тактах сигнала синхронизации. Конечные автоматы дают возможность увеличить тактовую частоту схемы за счёт разбиения вычисления на несколько тактов, а также реализовать ожидание внешних событий в течение произвольного времени. Практически любое нетривиальное управляющее устройство цифровой схемы содержит один или несколько конечных автоматов.

Несмотря на концептуальную простоту модели, реализация конечных автоматов на практике сопряжена с риском возникновения ошибок, которые трудно обнаружить традиционными методами верификации, основанными на моделировании. К таким ошибкам относятся недостижимые состояния (в которые автомат не может попасть ни при каком входном воздействии), мёртвые переходы (условие которых никогда не выполняется), дедлоки (блокировки) и лайвлоки («живые» блокировки). Покрытие всех подобных сценариев стандартным симуляционным тестированием в общем случае нетривиально, поскольку оно требует написания соответствующего тестового окружения.

Формальные методы верификации свойств (англ. Formal Property Verification, FPV) в принципе позволяют обнаруживать такие проблемы, однако требуют от инженеров - верификаторов ручного написания свойств и значительных трудозатрат, что ограничивает их применимость на ранних

стадиях проектирования. Методы статического анализа конечных автоматов дополняют FPV, обеспечивая автоматическое обнаружение структурных аномалий в графе автомата. Это делает их особенно удобными для проверок на ранних этапах проектирования и для интеграции в процессы непрерывной интеграции.

Данная работа посвящена разработке инструмента SVAN-FSM для автоматического обнаружения и анализа конечных автоматов в описаниях на языке SystemVerilog. Инструмент разработан в рамках проекта SVAN [3] — системы статического анализа SystemVerilog-описаний. В основе предложенного подхода лежит извлечение конечного автомата из списка логических элементов (netlist), получаемого после обработки исходного кода компилятором, построение графового представления автомата и применение к нему алгоритмов статического анализа для поиска типичных ошибок. Особое внимание уделяется сопоставлению результатов анализа с исходным кодом, что существенно улучшает интерпретируемость полученных данных для пользователя.

2 Постановка задачи

Цель работы: разработать инструмент для извлечения и анализа конечных автоматов из описаний аппаратуры на языке SystemVerilog

Основные задачи:

1. Исследовать модель конечного автомата в аппаратуре, различные способы её реализации и методы автоматического обнаружения конечных автоматов
2. Разработать инструмент для автоматического обнаружения и извлечения конечных автоматов и реализовать на основе открытой инфраструктуры синтеза yosys и библиотеки slang
3. Реализовать анализатор конечных автоматов с применением графовых методов для обнаружения типовых ошибок: мёртвых переходов, недостижимых состояний, блокировок, «живых» блокировок
4. Поддерживать сопоставление информации АСД (slang) и списка соединений (yosys) для получения полной информации об автомате
5. Проверить корректность разработанного инструмента путём тестирования на открытых проектах и сравнения с коммерческими аналогами

3 Обзор существующих решений

Существующие подходы к извлечению конечных автоматов из описаний на языках описания аппаратуры, подробный обзор которых приведён в [2], можно разделить на две крупные группы: подходы, работающие с абстрактным синтаксическим деревом (АСД) исходного кода, и подходы, работающие со списком логических элементов, полученным после синтеза или частичной обработки. Каждая из групп имеет свои преимущества и ограничения, которые определяют области их применения.

3.1 Извлечение конечных автоматов на основе АСД-представления

В языке SystemVerilog конечные автоматы чаще всего описываются с помощью блоков `case`, в которых метки соответствуют текущим состояниям автомата, а тела ветвей задают условия перехода и присваивания следующего состояния. АСД-ориентированный подход выполняет поиск таких блоков непосредственно в дереве разбора: анализатор ищет процедурный блок, содержащий оператор `case`, извлекает состояния из меток и восстанавливает переходы по внутренней логике. Этот подход интуитивно понятен и даёт прямое соответствие между результатами анализа и исходным кодом.

Однако в реальной практике АСД-подход сталкивается с рядом сложностей. Во-первых, не все конечные автоматы описываются с помощью канонического шаблона на основе `case`. Существуют альтернативные стили: автоматы, реализованные через вложенные конструкции `if/else` или с помощью набора непрерывных присваиваний. Каждому альтернативному стилю требуется отдельная эвристика обнаружения, и покрыть все встречающиеся варианты крайне трудно. Во-вторых, логика переходов может находиться в процедурных блоках как с последовательностной (`always_ff`, `always @(posedge clk)`), так и с комбинационной (`always_comb`, `always @*`) семантикой. В комбинационном случае вводится отдельная переменная следующего состояния, и анализ должен верифицировать, что переменные текущего и следующего состояния связаны последовательностной логикой — как правило, через присваивание в отдельном `always_ff` блоке. В-третьих, дополнительная логика переходов может располагаться до или после оператора `case` в том же процедурном блоке, например, проверка наличия глобальных сбоев в системе.

Основное достоинство АСД-подхода — прямое сопоставление результатов анализа с исходным кодом, так как всю необходимую информацию (имена переменных, номера строк, выражения) можно получить непосредственно из синтаксического дерева. Этот подход применён, в частности, в работе [4]. Авторы накладывают жёсткие ограничения на стиль описания конечных автоматов, упрощая реализацию анализатора. Такой подход эффективен при условии совместной работы разработчиков анализатора и проектировщиков RTL в рамках одной организации, однако он плохо подходит для универсального коммерческого статического анализатора, который должен корректно работать с разнородным кодом, полученным из разных источников.

3.2 Извлечение конечных автоматов на списке логических элементов

Альтернативный подход выполняет обнаружение и извлечение конечных автоматов не из АСД, а из списка логических соединений. Данную стратегию используют Yosys [5], AVFSM [6], FSMx [7], FSMx-Ultra [8] и работа Шеня и др. [9]. Несмотря на различия в деталях, все подходы на основе списка логических элементов обладают рядом общих характеристик.

Первая особенность состоит в том, что netlist унифицирует структурно схожие конструкции исходного языка. Так, конструкции if и case на уровне netlist сводятся к мультиплексорам с различным числом входов, что позволяет применять одинаковые алгоритмы обнаружения вне зависимости от исходного стиля кодирования.

Вторая особенность — в том, что функции переходов и выходов определяются через «выполнение» схемы в абстрактном смысле: для каждого возможного значения текущего состояния и каждой комбинации входных сигналов вычисляется следующее состояние и выход. Этот подход способен обрабатывать сколь угодно сложную структуру комбинационной логики, однако может игнорировать зависимости между входами и в худшем случае такой алгоритм имеет экспоненциальную сложность в зависимости от числа входов автомата.

Третья особенность связана с тем, что оптимизации, применяемые при синтезе (например, удаление недостижимого кода, слияние логических вентилей, распространение констант), могут оказать как положительное, так и отрицательное влияние на последующий анализ. Положительный

эффект проявляется в упрощении структуры netlist; отрицательный — в потенциальной потере информации о конструкциях исходного кода.

Четвёртая особенность — потенциальная потеря информации об исходном коде при переводе АСД в netlist. Это существенно ухудшает интерпретируемость результатов анализа для конечного пользователя: вместо имён состояний, определённых в исходном коде, пользователь видит абстрактные числовые коды; вместо осмысленных имён сигналов — служебные идентификаторы, сгенерированные компилятором.

Таблица 1: Placeholder Caption

Критерий	АСД-подход	Подход на основе netlist
Независимость от стиля кода	Низкая	Высокая
Связь с исходным кодом	Прямая	Требует доп. действия
Обработка распределённой логики	Ограниченная	Поддерживается
Параметризованные дизайны	В одном модуле	Для всего дизайна
Влияние оптимизаций синтеза	Не применимо	Может влиять

3.3 Методы анализа извлечённых моделей конечных автоматов

После того как конечный автомат извлечён из исходного описания, к полученной модели могут быть применены различные методы анализа. Эти методы условно разделяются на две группы: графовые методы и символьные методы.

Графовый анализ. Распространённый подход, при котором конечный автомат представляется в виде направленного графа состояний и переходов, после чего к нему применяются стандартные алгоритмы теории графов для поиска достижимости состояний, дедлоков и лайвлоков. Подобные методы широко применяются в верификации и проверке управляющей логики [10, 11].

Символьный анализ. Альтернативный класс методов представляет множества состояний и переходов в неявной форме, например, с помощью двоичных решающих диаграмм (англ. Binary Decision Diagrams, BDD) или кодирования через задачи выполнимости логических формул (SAT/SMT). Это позволяет проводить анализ достижимости и проверку свойств без явного перечисления всех состояний [12, 13]. Символьные методы часто применяются в системах проверки моделей (model checking) и особенно эффективны для автоматов с регулярной структурой и боль-

шим числом состояний. Недостатком таких методов является более высокая сложность реализации и необходимость грамотного выбора порядка переменных BDD или стратегии SAT-решателя для обеспечения приемлемой производительности.

3.4 Сравнение существующих инструментов анализа

В Таблице 2 приведено сравнение существующих инструментов анализа конечных автоматов по ряду ключевых характеристик. Под охватом обнаружения конечных автоматов понимается диапазон шаблонов, которые инструмент может успешно распознать. Инструменты с низким охватом распознают только канонические шаблоны (например, оператор `case` внутри `always_ff`), инструменты с высоким охватом обрабатывают также нетипичные паттерны: переходы на основе `if/else`, логику, распределённую по нескольким процедурным блокам, регистры состояния, вынесенные в отдельные модули, различные стили автоматов (Мур, Мили).

Инструмент AVFSM [6] ориентирован на обнаружение уязвимостей и отслеживание информационных потоков, но не выполняет структурного анализа автомата (поиск дедлоков, лайвлоков, недостижимых состояний). Инструменты FSMx [7] и FSMx-Ultra [8] работают на уровне списка соединений вентилях и предназначены для оценки безопасности через анализ достижимости состояний; они не поддерживают работу на уровне RTL и не предоставляют сопоставления с исходным кодом. Базовая реализация Yosys [5] обеспечивает только извлечение автоматов, но не их анализ на ошибки рассматриваемых классов; кроме того, она ограничена автоматами Мура и не предоставляет сопоставления с исходным кодом. Подход Ванга и Эдсолла [4] основан на поиске шаблонов в АСД и ограничен каноническими стилями `case-based` автоматов. Подход Шеня и др. [9] предлагает эффективный алгоритм извлечения на уровне списка соединений вентилях, но не решает задачи анализа на уровне RTL и сопоставления с исходным кодом. Подход Муна и Харера [14] использует абстракцию на основе BDD для проверки дедлоков и лайвлоков; метод ориентирован на вентильный уровень и требует заранее заданных свойств, а не выполняет полностью автоматическое извлечение и анализ.

Таблица 2: Сравнение существующих инструментов извлечения и анализа конечных автоматов

Возможность	Ванг [4]	Yosys [5]	AVFSM [6]	FSMx [7,8]	Шень [9]	Мун [14]
Входной уровень	АСД	RTL	Вентили	Вентили	Вентили	Вентили
Охват	Низкий	Средний	Высокий	Высокий	Высокий	Высокий
Связь с кодом	Прямая	Нет	Нет	Нет	Нет	—
Делдок/лайвлок	Нет	Нет	Нет	Нет	Нет	Да
Недостижимые	Да	Нет	Нет	Да	Нет	Нет
Мёртвые пер.	Нет	Нет	Нет	Нет	Нет	Нет
Анализ уязв.	Нет	Нет	Да	Да	Нет	Нет
Взаим. автоматы	Нет	Нет	Нет	Нет	Нет	Нет

4 Исследование и построение решения задачи

4.1 Конечные автоматы и возможные ошибки в них

4.1.1 Основные определения

Конечный автомат — шестёрка $(S, I, O, \delta, \lambda, s_0)$, где:

- S — конечное множество состояний автомата;
- I — множество входных сигналов;
- O — множество выходных сигналов;
- $\delta : S \times I \rightarrow S$ — функция переходов;
- λ — функция выходов (зависит от типа автомата);
- $s_0 \in S$ — начальное состояние.

Конечный автомат в RTL — последовательностная логическая модель, описывающая поведение системы через конечное множество дискретных *состояний*, *переходов* между этими состояниями и соответствующих им *выходов*. В любой момент времени автомат находится ровно в одном из своих возможных состояний. Следующее состояние определяется *функцией переходов*, которая зависит от *текущего состояния* и *входов* автомата. В RTL-коде состояние хранится в выделенном регистре, сами состояния кодируются именованными константами, а *входами* автомата служат сигналы, управляющие условиями переходов — выражениями в конструкциях if, case и тернарных операторах.

По виду функции выходов различают два классических типа конечных автоматов:

- **Автомат Мура** — выходы зависят только от текущего состояния. Выход определяется содержимым регистра состояния и не меняется на текущем такте при изменении входов. Формально это означает, что функция переходов имеет следующий вид:

$$\lambda : S \rightarrow O$$

- **Автомат Мили** — выходы зависят от текущего состояния и от текущих значений входов. Такая модель позволяет получать быстрый отклик на входные сигналы, но усложняет верификацию из-за комбинационного пути от входа к выходу. Функция переходов имеет вид:

$$\lambda : S \times I \rightarrow O$$

В настоящей работе рассматриваются автоматы обоих типов. Поскольку для обнаружения рассматриваемых видов ошибок (недостижимые состояния, мёртвые переходы, дедлоки, лайвлоки) функция выходов несущественна, различие между Муром и Мили в анализе сводится к единообразной обработке.

Автомат можно представить в виде ориентированного графа $G = (V, E)$, где:

- $V = S$ — множество вершин (состояний)
- $E \subseteq S$ — множество переходов

Каждому ребру $e = (s_i, s_j)$ сопоставляется булево условие перехода $cond_{ij}$, определённое над входами автомата I .

4.1.2 Конечные автоматы в языках описания аппаратуры

В языке SystemVerilog конечный автомат обычно реализуется через комбинацию регистра состояния и комбинационной или последовательностной логики, определяющей следующее состояние. Канонический шаблон описания выглядит следующим образом: вводится тип для состояний (обычно с помощью `typedef enum`), объявляется регистр состояния и описывается один или два `always`-блока для логики переходов. В односоставном варианте (одноблочный стиль) вся логика помещается в `always_ff`, и переходы

оформляются в виде оператора `case`. В двухсоставном варианте (двухблочный стиль) комбинационная логика вычисления следующего состояния выносится в `always_comb`, а сам регистр обновляется в отдельном `always_ff`.

На практике встречаются также варианты, в которых регистр состояния реализуется как экземпляр параметризованного модуля триггера (например, шаблон из ядра VeeR-EL2, где функция следующего состояния вычисляется в одном процедурном блоке, а регистр инстанцируется как модуль `rvdff`). Встречаются автоматы, в которых в одном `always`-блоке совмещаются логика сброса и логика переходов; автоматы с несколькими регистрами состояния; автоматы, реализованные через `if/else` вместо `case`. Эта вариативность стилей существенно затрудняет разработку универсального анализатора.

Дополнительную сложность вносят параметризованные модули: один и тот же фрагмент исходного кода может порождать конечные автоматы с различным набором состояний и переходов в зависимости от значений параметров, подставленных при инстанцировании. Корректный анализ таких автоматов требует работы на уровне `netlist`, полученного после параметризации (элаборации).

4.1.3 Классификация типичных ошибок в конечных автоматах

В реализации конечных автоматов в аппаратуре встречаются следующие типичные ошибки:

Недостижимое состояние — состояние, в которое автомат не может попасть из начального состояния ни при каком допустимом входном воздействии. Причины: опечатки в именах состояний, неверные условия переходов, либо неактуальный код, оставшийся после рефакторинга.

Мёртвый переход — переход, который никогда не происходит по каким-либо причинам (например, условие которого никогда не выполняется ни при каких входных воздействиях).

Мёртвые переходы можно разделить на 2 типа:

- Переходы, условия которых никогда не выполняются. Причинами таковых могут быть несовместные условия (например, выражение `ctl & !ctl` всегда ложно), сигналы, жёстко привязанные к константам параметризацией или оптимизацией, а также логические ошибки в вы-

ражениях условий.

- Переходы, происходящие из недостижимых состояний

В рамках анализа будет рассматриваться первый тип мёртвых переходов, вторые же будут обнаруживаться автоматически во время анализа недостижимых состояний.

Дедлок (блокировка) — состояние, из которого не существует ни одного перехода в другое состояние. В зависимости от проекта дедлок может быть как ошибкой, так и намеренно спроектированным завершающим состоянием (например, после обнаружения необратимой аварии).

Лайвлок («живая» блокировка) — группа состояний, переходы между которыми образуют замкнутый цикл, но ни одно из них не имеет перехода в состояние вне этой группы. Как и в случае дедлока, лайвлок может быть намеренным (например, финальная фаза многофазного автомата, которая не возвращается в начальное состояние).

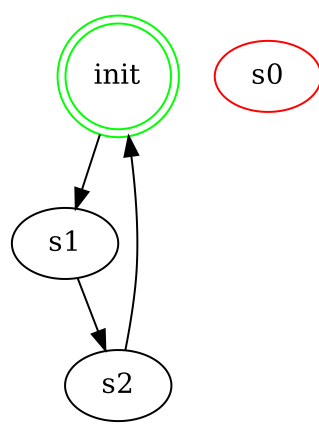
Автоматический анализатор должен уметь обнаруживать все четыре вида ошибок. При этом нужно понимать, что не каждый выявленный паттерн является реальной ошибкой — многие лайвлоки и дедлоки соответствуют намеренной логике. Это требует ручной проверки и классификации результатов пользователем.

4.1.4 Сильно связанные компоненты и их роль в анализе

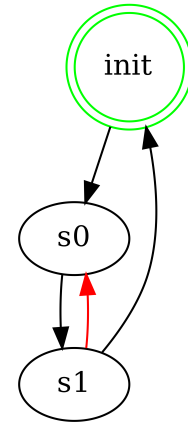
Сильно связной компонентой (ССК) графа называют максимальный его подграф, в котором из каждой вершины достижима любая другая.

Главной группой называется ССК, содержащая начальное состояние. Эта компонента представляет собой «рабочую часть» автомата — множество состояний, между которыми автомат может курсировать в обычной работе.

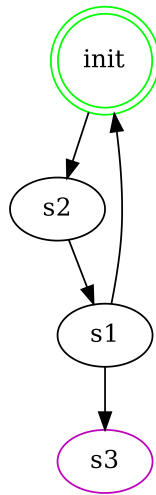
Терминальной сильно связной компонентой (ТССК) называется ССК, отличная от главной группы и не имеющая исходящих рёбер в вершины вне компоненты. ТССК соответствует ситуации, когда автомат, попав в одно из её состояний, больше никогда не сможет покинуть эту группу. Именно ТССК лежат в основе формального определения дедлоков



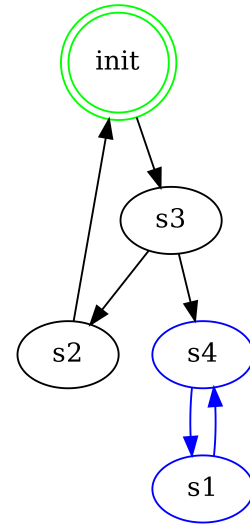
Недостижимое состояние (помечено красным)



Мёртвый переход (помечен красным)



Дедлок (помечен фиолетовым)



Лайвллок (помечен синим)

Таблица 3: Типы ошибок в конечных автоматах

и лайвлоков: ТССК из одной вершины есть дедлок, ТССК из нескольких вершин — лайвллок.

Для нахождения ССК применяется классический алгоритм Тарьяна, имеющий временную сложность $O(|V| + |E|)$, где $|V|$ и $|E|$ — число вершин и рёбер графа соответственно. Это позволяет выполнять анализ даже на больших автоматах за приемлемое время.

4.2 Инструмент SVAN и библиотека slang

SVAN [3] — это разрабатываемая в ИСП РАН система статического анализа для языка SystemVerilog. Проект направлен на создание промышленного анализатора, способного работать с реальными промышленными RTL-проектами, включая проекты, критичные с точки зрения безопасности.

Ключевым компонентом SVAN на этапе разбора исходного кода явля-

ется открытая библиотека slang [15] — инструментарий для работы с языком SystemVerilog на уровне АСД. Библиотека предоставляет полноценный лексический и синтаксический разбор, построение абстрактного синтаксического дерева, разрешение символов, проверку типов и элаборацию (раскрытие параметров, разворачивание иерархии модулей). Важной особенностью slang является способность обрабатывать реальный производственный код с поддержкой большинства конструкций стандарта SystemVerilog IEEE 1800-2017.

Внутреннее представление slang включает абстрактное синтаксическое дерево с подробной информацией о каждом узле, позицию каждого символа в исходных файлах (через объект `SourceLocation`), информацию о типах выражений и таблицу символов. Для задачи статического анализа конечных автоматов особенно важна возможность получать для каждого выражения точную ссылку на его местоположение в исходном коде — эта информация впоследствии используется при сопоставлении результатов анализа с исходным кодом.

4.3 Синтезатор yosys и его подход к извлечению конечных автоматов

4.3.1 Общая информация о Yosys

Yosys [5] — открытая инфраструктура для синтеза цифровых логических схем, предназначенная для преобразования RTL-описаний на Verilog или SystemVerilog в различные представления: от высокоуровневого списка соединений до вентильного уровня и файлов в форматах, поддерживаемых конкретными технологиями. Yosys реализует процесс синтеза в виде последовательности независимых проходов (*passes*), каждый из которых выполняет определённую трансформацию внутреннего представления. Такой архитектурный подход обеспечивает расширяемость: новые алгоритмы анализа и оптимизации могут быть добавлены как отдельные проходы, не затрагивая существующий код.

Внутреннее представление Yosys, называемое RTLIL (RTL Intermediate Language), представляет собой список логических элементов: проводов (*wires*), регистров (в том числе D-триггеров различных типов), мультиплексоров, логических вентилей, арифметических блоков и отдельных специальных элементов. RTLIL достаточно низкоуровневое, чтобы допускать большое количество оптимизаций, но при этом достаточно абстрактное, чтобы со-

хранять высокоуровневые конструкции (например, ячейка \$add или \$mux не обязательно разворачиваются сразу в вентили).

4.3.2 Представление логики переходов в виде мультиплексорных деревьев

Одним из ключевых аспектов работы синтезатора Yosys является приведение различных конструкций языка SystemVerilog к унифицированному низкоуровневому представлению. За это преобразование в предложенной схеме анализа отвечает библиотека yosys-slang.

Рассмотрим, как условные конструкции (например, `if/else` и `case`) преобразуются в сеть мультиплексоров. В общем случае любая конструкция ветвления может быть сведена к мультиплексору. Например, оператор:

```
if (cond)
    a = b;
else
    a = c;
```

эквивалентен мультиплексору `a = cond ? b : c`. Аналогично, оператор `case` преобразуется в каскад мультиплексоров с несколькими входами:

```
always_ff @(posedge clk) begin
    case (state)
        S0: state <= S1;
        S1: state <= S2;
        default: state <= S0;
    endcase
end
```

На уровне RTLIL данная логика преобразуется в комбинационную схему, формирующую значение следующего состояния на входе D-триггера. Эта схема представляется в виде дерева мультиплексоров, где:

- Входы мультиплексоров соответствуют возможным значениям следующего состояния;
- Управляющие сигналы кодируют текущие условия (например, равенство `state == S0`);
- Структура дерева отражает вложенность условных операторов.

Таким образом, функция переходов конечного автомата реализуется в виде комбинационной схемы, представленной как мультиплексорное дерево, подающееся на вход D-триггера, хранящего состояние.

4.3.3 Извлечение конечных автоматов в Yosys

Для задачи обнаружения и извлечения конечных автоматов в Yosys реализованы два прохода [17]:

fsm_detect — проход поиска кандидатов в конечные автоматы. Проход перебирает все D-триггеры в схеме и для каждого проверяет пару (триггер, выходной провод Q). Начиная с D-входа триггера, выполняется обход мультиплексорного дерева: если на всех его листьях находятся либо константы, либо тот же самый провод Q, то триггер считается кандидатом в регистр состояния. Выполняется ряд дополнительных проверок; например, в исходной реализации Yosys требуется, чтобы выходы автомата зависели только от текущего состояния, что ограничивает обнаружение автоматами Мура.

fsm_extract — проход собственно извлечения автомата. Он преобразует пару «триггер + мультиплексорное дерево» в специализированную ячейку \$fsm, инкапсулирующую всю логику автомата: состояния, таблицу переходов, информацию о сигналах тактирования и сброса [18]. Процедура включает два этапа: (1) определение входов автомата и возможных состояний обходом мультиплексорного дерева; (2) определение функций переходов и выходов путём «вычисления» схемы для каждого возможного значения состояния. Если вычисления недостаточно для однозначного определения следующего состояния, индивидуальные входы фиксируются в логический ноль или единицу, и процесс повторяется.

Хотя базовые проходы Yosys работоспособны и неплохо покрывают канонические стили конечных автоматов, они имеют ряд ограничений, существенно сказывающихся на реальных проектах. Во-первых, не поддерживаются автоматы Мили. Во-вторых, неточно обрабатывается синхронный сброс: для синхронно сбрасываемых регистров сигнал сброса попадает в число входов автомата, что приводит к появлению фиктивных переходов из каждого состояния в начальное и искажает последующий анализ дедлоков и лайвлоков. В-третьих, отсутствует поддержка случаев, когда регистр состояния встроен в более крупную структуру и на провод отображается не целиком.

В настоящей работе базовая схема (fsm_detect + fsm_extract) была сохранена, а каждый проход индивидуально расширен для устранения

описанных ограничений. Детали расширений рассматриваются в разделе 5.2.

4.3.4 Ограничения представления

Несмотря на универсальность, мультиплексорное представление имеет ряд особенностей:

- Оптимизации синтеза могут изменять структуру дерева (например, объединять ветви или удалять недостижимые пути);
- Условия переходов теряют явную семантику и должны восстанавливаться из логической структуры;
- При наличии сложной комбинационной логики глубина дерева может значительно увеличиваться.

Тем не менее, именно такое представление делает возможным унифицированный алгоритм извлечения конечных автоматов, реализованный в данной работе.

4.4 Библиотека yosys-slang

Связующим звеном между библиотекой slang и синтезатором Yosys является библиотека yosys-slang [16] — фронтенд, преобразующий АСД, построенное slang, во внутреннее представление RTLIL Yosys. Такая архитектура имеет несколько преимуществ по сравнению со встроенным в Yosys стандартным фронтендом Verilog.

Во-первых, slang обеспечивает существенно более точный разбор SystemVerilog, включая расширенные конструкции стандарта IEEE 1800-2017, которые не поддерживаются штатным фронтендом Yosys. Это расширяет спектр RTL-описаний, которые может обрабатывать конвейер обработки.

Во-вторых, yosys-slang обеспечивает непрерывную связь между АСД-уровнем (slang) и netlist-уровнем (RTLIL). Каждый элемент netlist может нести информацию о том, из какой конструкции исходного кода он был получен. Эта информация хранится в атрибутах RTLIL-ячеек и проводов. В дальнейшем атрибуты будут использоваться в работе для восстановления

контекста (имён состояний, выражений условий) при выводе результатов пользователю.

В рамках настоящей работы yosys-slang потребовал расширения: был изменён формат атрибута `src`, хранящего информацию о местоположении элемента в исходном коде. Подробности этих изменений изложены в разделе 5.4.

5 Описание практической части

5.1 Общая схема работы анализатора

Разработанный инструмент SVAN-FSM реализует сквозной процесс автоматического обнаружения, извлечения и анализа конечных автоматов из описаний на языке SystemVerilog. Общая схема работы инструмента представлена на Рисунке 1. Процесс состоит из следующих этапов:

1. **Построение АСД.** Исходный код на SystemVerilog разбирается библиотекой slang [15]. В результате строится абстрактное синтаксическое дерево, содержащее все конструкции исходного кода, информацию о типах, именах и положениях выражений.
2. **Трансляция в RTLIL.** АСД транслируется во внутреннее представление Yosys — RTLIL — с помощью yosys-slang [16].
3. **Обнаружение и извлечение автоматов.** Расширенная реализация проходов fsm_detect и fsm_extract выполняет поиск конечных автоматов в RTLIL-представлении, идентифицирует их входы, выходы, состояния и переходы, и извлекает каждый автомат в отдельную компактную ячейку \$fsm.
4. **Графовый анализ.** Каждый извлечённый автомат преобразуется в направленный граф с использованием библиотеки Boost Graph Library. Полученный граф последовательно проходит несколько стадий анализа, в ходе которых выявляются мёртвые переходы, недостижимые состояния, дедлоки и лайвлоки.
5. **Сопоставление с исходным кодом.** Устанавливается соответствие между элементами АСД (символами, выражениями) и элементами netlist (проводами, сигналами). Это соответствие позволяет восстановить имена состояний, условия переходов и ссылки на исходный код в граф-представлении автомата.

5.2 Извлечение конечных автоматов из списка логических элементов

Базовая схема извлечения автоматов, предложенная в Yosys (проходы fsm_detect и fsm_extract), была расширена для устранения ряда прак-

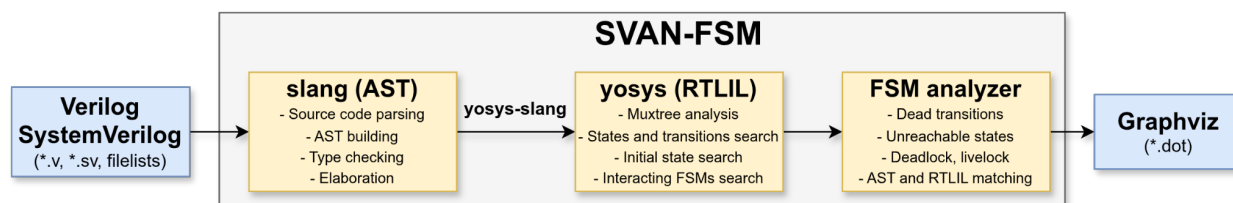


Рис. 1: Общая схема работы разработанного анализатора конечных автоматов

тических ограничений, сказывающихся на применимости инструмента в реальных RTL-проектах. Ниже описаны три ключевых расширения.

5.2.1 Разбиение проводов на части

Взаимно-однозначное соответствие между D-триггерами и проводами в netlist соблюдается не всегда. Рассмотрим типичный случай: в исходном коде объявлен массив состояний

```
state_t state_arr [N-1:0];
```

где каждый элемент массива содержит логику отдельного автомата. После преобразования в netlist для каждого элемента создаётся свой D-триггер, однако все триггеры могут разделять общий провод (например, если массив объявлен как общий регистр). Это приводит к отображению «многие к одному»: несколько триггеров отображаются на один и тот же провод. Аналогичная ситуация возникает, когда регистр состояния встроен в более крупную структуру (например, в packed struct или в более широкий регистр), и на провод приходится лишь фрагмент, соответствующий конкретному автомату.

Для решения этой задачи реализован механизм, названный разбиением проводов на части (wire blasting). При обнаружении триггера-кандидата в регистр состояния соответствующий фрагмент общего провода отделяется и заменяется новым, независимым проводом. Это восстанавливает взаимно-однозначное соответствие между триггером и проводом, после чего стандартный алгоритм извлечения работает корректно.

Реализация механизма проверяет, что используется именно фрагмент провода, а не весь провод целиком (иначе операция не нужна), и что после разделения сохраняются все входящие и исходящие связи. Операция разбиения выполняется один раз для каждого кандидата и не влияет на общую вычислительную сложность алгоритма.

5.2.2 Поддержка автоматов Мили

Исходная реализация Yosys ограничивается автоматами Мура: функция выходов может зависеть только от текущего состояния. Эта проверка выполняется в проходе `fsm_detect` и отбрасывает автоматы Мили, встречающиеся во многих реальных проектах.

Расширение анализатора для поддержки Мили основано на следующем наблюдении: для обнаружения рассматриваемых видов ошибок (мёртвые переходы, недостижимые состояния, дедлоки, лайвлоки) функция выходов несущественна — анализу подлежит только граф переходов между состояниями. Поэтому при обработке кандидата в автомат Мили мы отбрасываем зависимые от входов выходы, сохраняя лишь переходы между состояниями. Это позволяет обрабатывать все автоматы единообразно — как Мура — и одновременно повышает производительность (меньше данных для обработки).

Модификация реализована на уровне прохода `fsm_detect`: условие, требовавшее исключительной зависимости выходов от состояния, заменено на фильтрацию выходов с последующим их удалением из `$fsm`-ячейки. Функции переходов при этом строятся, как и для автомата Мура, через перебор состояний и «вычисление» комбинационной логики.

5.2.3 Обработка синхронного сигнала сброса

В Yosys синхронные и асинхронные регистры обрабатываются по-разному. Для асинхронных регистров сброс реализуется специальной ячейкой `$adff[TODO]`, в которой сигнал сброса передаётся отдельным входом. Это позволяет алгоритму обнаружения автомата исключить сигнал сброса из рассмотрения и корректно определить начальное состояние (значение, в которое регистр переходит при активном сбросе). Для синхронных регистров сброс встроен в мультиплексорное дерево, подаваемое на D-вход обычной ячейки D-триггера. В этом случае сигнал сброса попадает в число входов автомата, что вызывает несколько проблем:

- Появляются дополнительные переходы из каждого состояния в начальное, активируемые сигналом сброса.
- Число входов автомата искусственно увеличивается на единицу, что усложняет последующий анализ.

- Нетривиальным становится определение самого начального состояния, поскольку Yosys в общем случае не решает эту задачу.

Последняя проблема особенно важна, поскольку без знания начального состояния невозможно проверять достижимость состояний и наличие дедлоков/лайвлоков.

Для корректной обработки синхронного сброса предложен следующий алгоритм. При обходе мультиплексорного дерева, начиная от корневого мультиплексора, подсчитывается число достижимых состояний в каждом поддереве. Если ровно одно из поддеревьев корневого мультиплексора содержит единственное состояние, то это состояние интерпретируется как начальное, а ветвь управления, активирующая соответствующий вход мультиплексора, — как сигнал сброса. Оставшаяся ветвь становится новым корнем мультиплексорного дерева, и дальнейший анализ уже не зависит от сигнала сброса.

Алгоритм гарантирует корректное восстановление начального состояния в широком классе реальных RTL-описаний и приводит обработку синхронно сбрасываемых регистров к тому же виду, что и асинхронных.

Сложность процедуры извлечения автомата определяется в первую очередь построением таблицы переходов в проходе `fsm_extract`. Для каждого из $|S|$ возможных состояний схема вычисляется путём перебора комбинаций входов до тех пор, пока все переходы не будут определены. В худшем случае это даёт сложность $O(|S| \cdot 2^{|I|} \cdot |M|)$, где $|I|$ — число входов автомата, $|M|$ — размер мультиплексорного дерева. На практике, однако, $|I|$ обычно невелико (порядка десятков), а ранние завершения при определении переходов существенно сокращают число требуемых итераций.

5.3 Графовый анализ конечного автомата

После извлечения каждый конечный автомат преобразуется в графовое представление и последовательно подвергается трём стадиям анализа. Каждая стадия уточняет граф, удаляя из него заведомо некорректные элементы, что повышает точность последующего анализа. На Рисунке 2 показан модельный пример графа автомата с размеченными ошибками и соответствующий ему исходный код на Verilog.

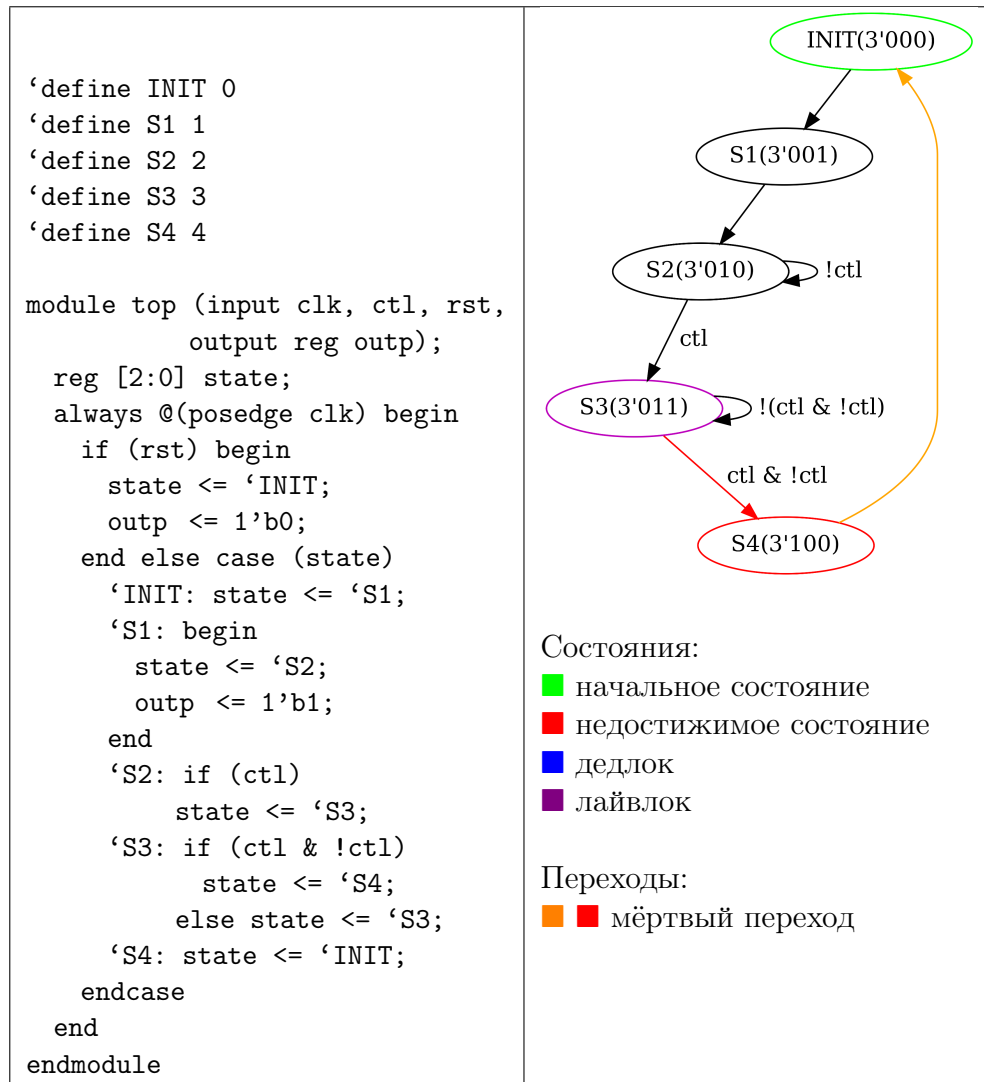


Таблица 4: Граф конечного автомата с размеченными ошибками (справа) и соответствующий ему Verilog-код (слева)

5.3.1 Обнаружение мёртвых переходов

На первой стадии анализа выполняется поиск мёртвых переходов. В текущей реализации обнаружение основано на выявлении противоречий, связанных с входами автомата, имеющими константные значения. Константные значения входов могут возникать по нескольким причинам: явное присваивание константы в исходном коде, значения параметров, подставленные при элаборации модуля, или результат оптимизаций синтеза (например, распространение констант, удаление недостижимых ветвей).

Алгоритм работает следующим образом. Сначала при обходе netlist определяются все входы автомата, жёстко привязанные к логическим 0 или 1. Затем для каждого перехода его условие реконструируется как конъюнкция атомарных предикатов над входами. Если хотя бы один из предикатов

противоречит известному константному значению входа (например, условие требует значения '1', тогда как вход постоянно равен '0'), переход классифицируется как мёртвый и удаляется из графа.

Несмотря на простоту, метод эффективен на практике: в реальных проектах константные входы возникают повсеместно, особенно в параметризованных модулях и после распространения констант при оптимизации. Удаление мёртвых переходов существенно повышает точность последующего анализа достижимости и сильно связанных компонент.

Более точное определение недостижимости переходов требует рассуждений над отношениями между непостоянными входами, что в общем случае эквивалентно задаче удаления мёртвого кода и является вычислительно дорогостоящим. Интеграция методов SAT-решения или символьного исполнения оставлена для будущих работ.

5.3.2 Анализ недостижимых состояний

На второй стадии выявляются недостижимые состояния. Недостижимые состояния определяются поиском в ширину (англ. breadth-first search, BFS) по графу, стартующим из начального состояния. Состояние классифицируется как недостижимое, если в него невозможно попасть ни по какому пути из начального состояния.

Существенно, что анализ работает на графе, из которого уже удалены мёртвые переходы. Это повышает число обнаруживаемых проблем и общую точность анализа: состояние, достижимое лишь через мёртвый переход, после удаления этого перехода окажется полностью изолированным и будет корректно классифицировано как недостижимое. В противном случае такое состояние ошибочно считалось бы достижимым.

5.3.3 Обнаружение дедлоков и лайвлоков

На третьей стадии обнаруживаются дедлоки и лайвлоки. Анализ основан на нахождении сильно связанных компонент (ССК) с помощью алгоритма Тарьяна, имеющего временную сложность $O(|V| + |E|)$, где $|V|$ и $|E|$ — число вершин и рёбер графа соответственно. Следуя определениям, данным в разделе 4.1.4, каждая найденная ССК проверяется на наличие исходящих рёбер, ведущих в вершины вне компоненты. Если таких рёбер

нет и компонента не является главной группой (содержащей начальное состояние), она классифицируется как ТССК.

Анализ выполняется на графе, из которого уже удалены мёртвые переходы и недостижимые состояния, что обеспечивает корректность результатов: все оставшиеся компоненты заведомо достижимы из начального состояния. ТССК, содержащая единственную вершину, классифицируется как дедлок; ТССК, содержащая несколько вершин, — как лайвлок.

Результаты анализа экспортируются в файл формата DOT (Graphviz) с разметкой: начальное состояние выделяется зелёным контуром, недостижимые — фиолетовым, дедлок — красным, лайвлок — оранжевым (см. Рисунок 2). Условия переходов выводятся в виде формулы в дизъюнктивной нормальной форме, реконструированной из атомарных входов автомата.

5.4 Сопоставление результатов анализа с исходным кодом

Для повышения удобства использования инструмента каждый конечный автомат представляется в виде направленного графа, в котором вершины — это состояния, а рёбра — переходы. Вершины помечаются именами состояний, заданными в исходном коде (как правило, через `enum` или `parameter`), а рёбра — условиями переходов, выраженными через оригинальные имена сигналов и констант. Поскольку на уровне `netlist` информация об исходном коде частично теряется, требуется специальная процедура сопоставления элементов АСД и `netlist`. Эта процедура реализована в виде двух последовательных проходов.

5.4.1 Отображение входов конечного автомата на выражения АСД

Первый проход, `fsm_match_conditions`, устанавливает соответствие между входами извлечённого конечного автомата и выражениями из АСД. Для обеспечения такого соответствия был существенно изменён формат атрибута `src` в RTLIL, в котором хранится информация о местоположении элементов в исходном коде.

Первое изменение касается способа хранения позиции. Поскольку библиотека `slang` не позволяет восстановить объект `SourceLocation` по имени файла и текстовой позиции, в новом формате `src` хранится пара: `BufferId` (идентификатор буфера, в котором хранится файл) и `Offset` (смещение в

байтах относительно начала буфера) — с помощью которой однозначно восстанавливается AST-символ, соответствующий данному проводу.

Второе изменение — поддержка посимвольной разметки. Один и тот же провод может соответствовать нескольким АСД-символам (например, если он образован из полей упакованной структуры). Поэтому новый формат `src` хранит информацию о местоположении не только для всего провода целиком, но и для отдельных его битовых срезов. Это позволяет корректно сопоставлять вход автомата с конкретным полем структуры в исходном коде.

Третье изменение — распространение `src` при соединениях. Когда в `netlist` соединяются два провода (например, при инстанцировании модуля), их атрибуты `src` наследуются друг от друга. Это обеспечивает непрерывность связи с исходным кодом даже сквозь границы модулей и промежуточные сигналы.

В совокупности эти изменения позволяют реализовать обход АСД, который собирает все условные конструкции (`if`, тернарные операторы, блоки `case`) и сопоставляет их местоположения с местоположениями входов автомата. В результате каждому входу автомата ставится в соответствие исходное выражение, что позволяет в дальнейшем выводить условия переходов в осмысленном виде.

5.4.2 Восстановление имён состояний

Второй проход, `fsm_analyze`, восстанавливает имена состояний. Задача эта нетривиальна, поскольку в реальных автоматах логика часто распределена по нескольким переменным: например, отдельно объявляется регистр состояния (`state`) и переменная следующего состояния (`next_state`), причём присваивание из `next` в `current` происходит в отдельном `always_ff` блоке или через порт инстанцируемого модуля триггера.

Алгоритм восстановления имён состоит из трёх шагов:

1. Для каждого АСД-символа собираются именованные константы, которые ему присваиваются или с которыми он сравнивается, а также другие символы, присваиваемые ему (в том числе через порты модулей).
2. Итеративно выполняется распространение присваиваний: каждый сим-

вол расширяет своё множество ассоциированных констант и символов, добавляя множества от всех символов, на которые он ссылается. Алгоритм конечен и сходится за число шагов, не превышающее размера графа ссылок.

3. Для каждого извлечённого конечного автомата соответствующий ему АСД-символ восстанавливается по иерархическому имени, и имена состояний берутся из полученного множества констант.

Между `fsm_match_conditions` и `fsm_analyze` запускаются оптимизационные проходы Yosys (`opt`). Эти оптимизации могут увеличить число константно-значных входов автомата и тем самым раскрыть дополнительные мёртвые переходы. Однако оптимизации могут также отбрасывать метаданные о местоположении в исходном коде. Двухпроходная архитектура решает эту проблему: `fsm_match_conditions` фиксирует все необходимые сопоставления до оптимизаций, после чего потеря части `src`-информации уже не влияет на качество анализа.

5.5 Обнаружение взаимодействующих автоматов

В реальных RTL-проектах управляющая логика редко реализуется в виде одного изолированного конечного автомата. Чаще встречается сценарий, при котором несколько автоматов взаимодействуют для реализации протоколов, конвейеров или цепочек управления. При этом ошибки в таких системах часто возникают не в отдельных автоматах, а именно во взаимодействии между ними. Это мотивирует разработку автоматического обнаружения зависимостей между конечными автоматами.

Формально мы определяем два автомата как взаимодействующие, если сигнал, генерируемый одним автоматом, достигает логики переходов другого автомата через путь, содержащий произвольную комбинационную логику и не более одного регистра.

Для обнаружения таких взаимодействий анализатор выполняет обход вперёд, начиная от каждого выхода конечного автомата. В процессе обхода допускается прохождение через любое число комбинационных элементов, тогда как через триггеры разрешается проходить не более одного раза. Всякий раз, когда достигается вход другого автомата, между соответствующими экземплярами автоматов фиксируется направленная зависимость.

Обход имеет линейную по размеру netlist сложность и вносит пренебрежимо малые накладные расходы по сравнению с извлечением автомата.

Обнаружение взаимодействующих автоматов даёт несколько важных преимуществ. Во-первых, оно помогает разработчику понять архитектурные управляющие зависимости, что особенно ценно в больших проектах, содержащих десятки автоматов. Во-вторых, информация о взаимодействиях может комбинироваться с диагностикой на уровне отдельных состояний: например, условие дедлока или лайвлока в одном автомате может распространяться на зависимые автоматы, что позволяет локализовать первопричину ошибки в одном блоке управления и её проявления в других. В-третьих, циклические зависимости в графе взаимодействий могут указывать на архитектурные проблемы, такие как непреднамеренная обратная связь между контроллерами.

6 Тестирование и результаты

6.1 Набор тестовых проектов

Разработанный инструмент был протестирован на синтетических тестах и на реальных открытых проектах аппаратного обеспечения. В тестовый набор вошли следующие проекты:

- Процессорные ядра архитектуры RISC-V: BlackParrot [20], CVA6 [21], Ibex [22], OpenC910 [23], PicoRV32 [24], SCR1 [25], VeeR-EH1 [26], VeeR-EL2 [27].
- Аппаратный блок шифрования AES [28].
- Открытая платформа корня доверия OpenTitan [29].

Для каждого из проектов подготавливался сценарий запуска, собирающий иерархию синтезируемой части проекта. Сценарии запуска стандартизованы и включают в себя настройку параметров сборки, указание списков файлов и определение макросов. Результаты работы инструмента воспроизводимы при повторном запуске в рамках одной и той же версии проекта.

6.2 Результаты обнаружения

Анализатор продемонстрировал корректную работу на всех оцениваемых проектах. В тестовом наборе были зафиксированы случаи всех четырёх анализируемых типов ошибок (см. Таблицу 3).

Проект	Автоматы	Мёртвые переходы	Недостиж. сост.	Дедлоки	Лайвлоки
AES	4	0	0	0	0
BlackParrot	18	3	3	3	5
OpenC910	1201	541	111	48	4
CVA6	24	13	5	0	0
Ibex	4	1	0	0	1
OpenTitan	592	71	58	95	14
PicoRV32	2	1	1	0	0
SCR1	26	0	0	0	0
VeeR-EH1	15	0	0	0	0
VeeR-EL2	12	0	0	0	0

Таблица 5: Caption

6.3 Классификация найденных проблем

Ручная верификация полученных результатов показала, что обнаруженные сценарии могут быть разделены на три категории:

Конфигурационные артефакты. Большинство мёртвых переходов и недостижимых состояний связаны с параметризованными модулями, инстанцированными с определёнными значениями параметров. Когда параметр отключает некоторую функциональность, соответствующие ветви в логике перехода становятся заведомо мёртвыми. Такие срабатывания не являются ошибками в прямом смысле слова, но их обнаружение полезно — оно показывает, какая часть кода «мертва» при данной конфигурации и, соответственно, может быть исключена при анализе или оптимизирована при синтезе.

Намеренное терминальное поведение. Многие из обнаруженных лайвблоков соответствуют многофазным автоматам, в которых фаза инициализации однократно переходит в основную фазу выполнения, никогда не возвращаясь в начальное состояние. Согласно графовому определению, это ТССК с более чем одной вершиной, и классифицируется как лайвлок. Однако в проектировании это закономерное поведение: автомат выполняет инициализацию один раз и дальше работает в установившемся режиме. Аналогично, некоторые дедлоки соответствуют терминальным состояниям обработки ошибок, из которых восстановление не предусмотрено намеренно (например, состояние «аппаратная блокировка» после обнаружения необратимого сбоя). Инструмент помечает такие случаи для ручной проверки, позволяя разработчику подтвердить намеренность.

Потенциальные ошибки. Меньшая часть обнаруженных аномалий представляет действительные ошибки, требующие исправления. Так, в проекте OpenC910 мёртвые переходы, вызванные константно-значными сигналами, привели к выявлению фрагмента кода, который никогда не исполняется; соответствующая проблема была оформлена и передана в основной репозиторий проекта [30].

Такая классификация характерна для результатов статического анализа в целом: инструмент не может самостоятельно различить намеренную и непреднамеренную логику, но предоставляет разработчику структурированное представление для быстрого принятия решений. Для облегчения

этого процесса результаты каждого анализа выводятся в формате DOT-графа с аннотациями, восстановленными из исходного кода.

6.4 Производительность

Для оценки вычислительных характеристик предложенного подхода было измерено сквозное время анализа (включая разбор, трансляцию в RTLIL, извлечение автоматов и графовый анализ) на репрезентативном подмножестве проектов. Измерения проводились на системе AMD Ryzen 5 4600G (6 ядер, 12 потоков, частота до 4,3 ГГц) с 16 ГБ оперативной памяти. Результаты представлены в Таблице 4.

Таблица 6: Сквозное время анализа для выбранных проектов

Проект	Строк кода	Автоматы	Время выполнения
PicoRV32	7,8 тыс.	2	0,6 с
BlackParrot	148 тыс.	18	27 с
OpenC910	406 тыс.	1201	121 мин

Время анализа изменяется в широких пределах: от долей секунды для небольших проектов (PicoRV32) до 27 секунд для проекта среднего размера (BlackParrot) и примерно двух часов для самого крупного (OpenC910, более 1200 автоматов на 406 тыс. строк кода).

Для выявления узкого места масштабируемости в Таблице 5 приведено разбиение времени выполнения по фазам для проекта OpenC910. Фаза графового анализа (включая обнаружение дедлоков, лайвлоков, недостижимых состояний и восстановление имён) завершается менее чем за две минуты (2% от общего времени) даже для этого самого крупного теста. Основные затраты приходятся на извлечение автоматов (55%) и их обнаружение (34%), что в сумме составляет 89% общего времени. Это согласуется с обсуждавшейся в разделе 5.2 худшей сложностью процедуры извлечения.

Таблица 7: Разбиение времени выполнения по фазам для проекта OpenC910 (общее время 121 мин)

Фаза	Время	Доля
Извлечение автоматов (fsm_extract)	67 мин	55%
Обнаружение автоматов (fsm_detect)	41 мин	34%
Сопоставление с кодом (fsm_match_conditions)	5 мин	4%
Разбор и трансляция в RTLIL	3 мин	2%
Графовый анализ и восстановление имён (fsm_analyze)	2 мин	2%
Оптимизационные проходы (opt)	3 мин	3%

Таким образом, узким местом масштабируемости является исчерпывающий перебор состояний при извлечении автомата, а не предложенные в работе алгоритмы анализа. Предложенная в работе сложность обеспечивает практически приемлемые времена выполнения для интеграции в рабочие процессы верификации — в частности, в ночные регрессионные прогоны или проверки перед коммитом.

6.5 Обсуждение проекта OpenC910

Проект OpenC910 заслуживает отдельного обсуждения. Анализатор обнаружил в нём 1201 экземпляр автомата, из которых только 88 являются уникальными (соответствуют различным определениям автоматов в исходном коде). Остальные экземпляры — это дубликаты, возникшие при элаборации, поскольку анализатор работает на уровне netlist, где для каждого экземпляра модуля создаётся отдельный набор логических элементов. Такое дублирование также затрагивает и сообщения об ошибках: одна и та же проблема в одном определении автомата повторяется для всех его экземпляров.

Для сравнения: базовая реализация Yosys (версия 0.58) без наших расширений обнаружила в том же проекте только 446 экземпляров (43 уникальных). Таким образом, предложенный инструмент выявляет примерно вдвое больше уникальных автоматов (88 против 43), что объясняется расширениями, описанными в разделе 5.2, — в первую очередь, разбиением проводов на части, поддержкой автоматов Мили и обработкой синхронного сигнала сброса. Эти расширения позволяют обнаруживать автоматы, которые немодифицированный Yosys распознать не в состоянии.

В этом проекте было обнаружено значительное число мёртвых переходов, и соответствующие проблемы переданы в основной репозиторий проекта. Как упоминалось в подразделе 6.3, переменная `ase_cfg`, жёстко привязанная к нулю, является характерным примером конфигурационного артефакта, приводящего к появлению мёртвых переходов.

6.6 Сравнение с коммерческим инструментом

Для синтетических тестов и проекта PicoRV32 выполнено сравнение с коммерческим инструментом статического анализа по точности извлечения автоматов и обнаружения ошибок. Результаты показали, что разрабо-

танный инструмент совпал с коммерческим аналогом во всех стандартных случаях и, сверх того, обнаружил существенно больше экземпляров автоматов в случаях, когда логика переходов выходит за рамки оператора case — например, когда блок case и связанная с ним логика сброса объединены в одном блоке always.

Особенно показателен случай, обнаруженный в процессорном ядре VeeR-EL2. В этом проекте используется характерный шаблон, когда комбинационная логика вычисления следующего состояния автомата (с помощью оператора case внутри блока always_comb) располагается в одном процедурном блоке, а регистр состояния реализуется как отдельный параметризованный модуль триггера (rvdff), подключаемый через порты модуля. Упрощённая версия этого шаблона приведена ниже.

```
module top (input logic clk, rst_l, en);
    typedef enum logic [2:0]
        {IDLE, START_WAIT, STOP} state_t;
    state_t buf_state, buf_nxtstate;

    always_comb begin
        buf_nxtstate = IDLE;
        case (buf_state)
            IDLE:      buf_nxtstate = START_WAIT;
            START_WAIT: buf_nxtstate = STOP;
            STOP:      buf_nxtstate = IDLE;
        endcase
    end

    rvdff #(.WIDTH($bits(state_t)))
        buf_state_ff (.din(buf_nxtstate),
                     .dout(buf_state),
                     .clk(clk), .rst_l(rst_l));
endmodule

module rvdff #(parameter WIDTH=1)
    (input logic [WIDTH-1:0] din,
     input logic clk, rst_l,
     output logic [WIDTH-1:0] dout);
    always_ff @(posedge clk or negedge rst_l)
        if (!rst_l) dout <= 0;
        else      dout <= din;
endmodule
```

Этот шаблон фактически распределяет конечный автомат по границам модулей: комбинационный always_comb блок вычисляет следующее состояние, а параметризованный триггерный модуль хранит текущее. Коммерческий инструмент не распознал такой шаблон как автомат, поскольку

ку АСД-ориентированное обнаружение ожидает, что регистр состояния и логика переходов находятся в одном и том же модуле. Предложенный в работе подход, основанный на netlist, корректно определил автомат, так как после элаборации и трансляции в RTLIL инстанцирование триггера уже разрешено, и мультиплексорное дерево напрямую связано с триггером — независимо от исходной иерархии модулей. Этот пример наглядно демонстрирует преимущества подхода, работающего с уровнем netlist, и показывает практическую ценность расширений, реализованных в настоящей работе.

6.7 Обработка параметризованных модулей

Дополнительно следует отметить важность корректной обработки параметризованных автоматов. В современных проектах на SystemVerilog часто используются параметризованные модули, в которых один и тот же фрагмент кода может порождать автоматы с различающимся поведением в зависимости от значений параметров. Анализ на уровне модуля без учёта конкретных параметров в общем случае недостаточен.

Поскольку разработанный инструмент работает на уровне netlist, генерируемого после элаборации параметров, каждый экземпляр автомата анализируется независимо со своими конкретными значениями параметров. Это позволяет обнаруживать конфигурационно-зависимое поведение, включая состояния и переходы, достижимые в одном экземпляре, но недостижимые в другом.

Такие эффекты были зафиксированы в ходе экспериментального исследования. Например, в модуле `br_uce` [19] проекта BlackParrot разные настройки параметров порождали разные графы автомата: в одних конфигурациях некоторые состояния оказывались недостижимыми. Это иллюстрируется на Рисунке 3, где показаны два экземпляра одного и того же автомата с различными наборами параметров. Кроме того, в ряде автоматов наблюдалась многофазная работа, состоящая из фазы инициализации и основной фазы выполнения, которая никогда не возвращается в начальное состояние; такое поведение, будучи намеренным, тем не менее классифицируется как лайвлок по графовому определению и помечается для ручной проверки.

Работа на уровне netlist, полученного после элаборации, позволяет

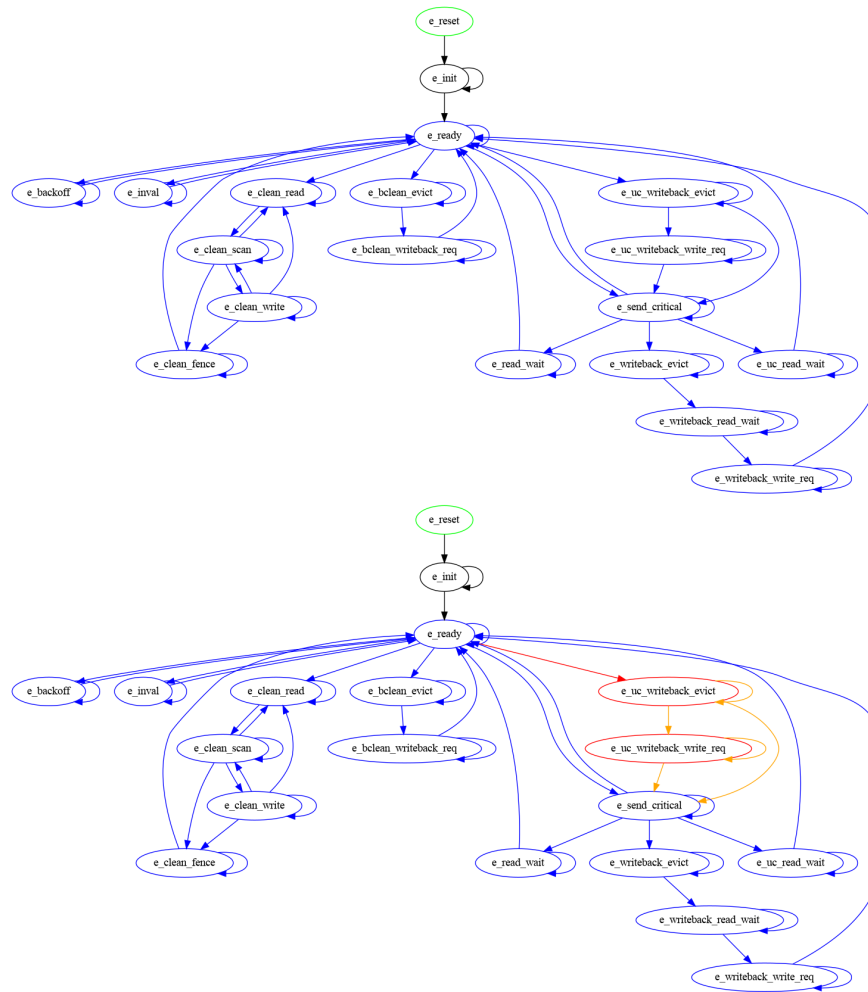


Рис. 2: Два экземпляра одного и того же автомата с разными значениями параметров в модуле br_uce проекта BlackParrot: (вверху) без недостижимых состояний, (внизу) с недостижимыми состояниями

разработанному методу улавливать конфигурационно-зависимое поведение и выявлять сценарии, которые были бы невидимыми при анализе на уровне исходного кода или отдельных модулей.

7 Заключение

В настоящей работе разработан инструмент SVAN-FSM — автоматический анализатор конечных автоматов в описаниях на языке SystemVerilog, созданный как часть системы статического анализа SVAN. Инструмент выявляет четыре класса проблем, связанных с конечными автоматами: мёртвые переходы, недостижимые состояния, дедлоки и лайвлоки. Дополнительно обнаруживаются взаимодействующие автоматы, что позволяет выявлять глобальные зависимости между блоками управления. Результаты анализа представляются в виде DOT-графов, связанных с исходным ко-

дом.

Работая на уровне netlist, анализатор сочетает графовый анализ с сопоставлением результатов с исходным кодом. Это обеспечивает надёжную обработку самых разных стилей написания RTL-кода, включая случаи, когда логика автомата распределена по нескольким процедурным блокам или даже по модульным границам. В инфраструктуру конечных автоматов Yosys внесены несколько существенных улучшений: поддержка синхронного сигнала сброса, разбиение проводов на части, единообразная обработка автоматов Мура и Мили.

Экспериментальное исследование на 10 открытых проектах аппаратного обеспечения, включая процессорные ядра RISC-V и критичные по безопасности проекты, продемонстрировало применимость подхода к промышленным масштабам. Время анализа остаётся приемлемым даже для самых крупных тестовых проектов: для OpenC910 (406 тыс. строк кода, 1201 автомат) полный анализ занимает около двух часов на стандартной рабочей станции.

Сравнение с коммерческим статическим анализатором показало, что предложенный инструмент значительно превосходит аналог по охвату обнаружения — в первую очередь в проектах с нетиповыми шаблонами реализации, например в случаях, когда регистр состояния вынесен в отдельный модуль.

Одним из текущих ограничений инструмента является использование эвристики распространения констант для обнаружения мёртвых переходов. Дальнейшие работы будут направлены на интеграцию более точных методов, таких как рассуждения на основе SAT-решения и символьное исполнение, для обнаружения мёртвых переходов, включающих непостоянные входы.

Другие запланированные направления включают:

- Расширение анализа взаимодействия автоматов для обнаружения глобальных сценариев дедлоков (дедлоков уровня системы, а не отдельного автомата).
- Интеграцию правил кодирования STARC, охватывающих типовые рекомендации по стилю описания автоматов на SystemVerilog.

- Расчёт дополнительных метрик автоматов: цикломатической сложности графа, стиля кодирования состояний (двоичный, Грея, one-hot), глубины графа, числа неиспользованных битов регистра состояния.
- Интеграция с пользовательским интерфейсом SVAN для более удобной навигации между результатами анализа и исходным кодом.

Список литературы

- [1] IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language. IEEE Std 1800-2017.
- [2] С.А., Смоллов. Обзор методов извлечения моделей из HDL-описаний // *Труды Института системного программирования РАН*. — 2015. — 01. — Vol. 27. — Pp. 97–124.
- [3] Чуркин Я.А., Бучацкий Р.А., Китаев К.Н., Волохов А.Г., Долгодворов Е.В., Камкин А.С., Коцыняк А.М., Самоваров Д.О. Система статического анализа для языка описания аппаратуры SystemVerilog // *Труды Института системного программирования РАН*. 2025;37(1):7-40. — 2025. — 03. — Vol. 37. — Pp. 7–40.
- [4] Wang T.-H., Edsall T. Practical FSM analysis for Verilog // *Proc. Int. Verilog HDL Conf. and VHDL Int. Users Forum, Santa Clara, CA*. — 1998. — P. 52–58.
- [5] YosysHQ. Yosys Open SYnthesis Suite. — <https://yosyshq.readthedocs.io/projects/yosys/en/latest>. — 2026. — accessed Feb. 2026.
- [6] Adib Nahiyani, Kan Xiao, Kun Yang, Yier Jin, Domenic Forte and Mark Tehranipoor. AVFSM: A framework for identifying and mitigating vulnerabilities in FSMs // *Proc. 53rd DAC*. — 2016. — P. 1–6.
- [7] R. Kibria, N. Farzana, F. Farahmandi, and M. Tehranipoor. FSMx: Finite state machine extraction from flattened netlist with application to security // *Proc. IEEE 40th VTS*. — 2022. — P. 1–7.
- [8] R. Kibria, F. Farahmandi, and M. Tehranipoor. FSMx-Ultra: Finite state machine extraction from gate-level netlist for security assessment // *IEEE Trans. CAD, vol. 42, no. 11*. — 2023. — 11. — P. 3613–3627.
- [9] Y. Shi, C. W. Ting, B.-H. Gwee, and Y. Ren. A highly efficient method for extracting FSMs from flattened gate-level netlist // *Proc. IEEE ISCAS*. — 2010. — P. 2610–2613.

- [10] *Clarke, E.M. and Grumberg, O. and Peled, D. and Peled, D.A.* Model Checking / Clarke, E.M. and Grumberg, O. and Peled, D. and Peled, D.A. The Cyber-Physical Systems Series. — MIT Press, 1999. <https://books.google.ru/books?id=Nmc4wEaLXFEC>.
- [11] *Hopcroft, John E.* Introduction to Automata Theory, Languages, and Computation / John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman. — Third edition. — Boston: Pearson/Addison Wesley, 2007. <http://infolab.stanford.edu/~ullman/ialc.html>.
- [12] *Bryant, R. E.* Graph-based algorithms for Boolean function manipulation // *IEEE Trans. Comput.*, vol. 35, no. 8. — 1986. — P. 677–691.
- [13] *Biere, A.* Handbook of Satisfiability / A. Biere. Frontiers in artificial intelligence and applications. — IOS Press, 2009. <https://books.google.ru/books?id=YVSM3sxhBhcC>.
- [14] *Moon, I.-H.* Abstraction-based livelock/deadlock checking for hardware verification. — 2012. — 10.
- [15] *Popoloski, M.* slang — SystemVerilog Language Services. — <https://github.com/MikePopoloski/slang>. — 2026. — accessed Feb. 2026.
- [16] yosys-slang: SystemVerilog frontend for Yosys. — <https://github.com/povik/yosys-slang>. — 2026. — accessed Feb. 2026.
- [17] *YosysHQ.* Yosys fsm_detect pass. — https://yosyshq.readthedocs.io/projects/yosys/en/latest/cmd/fsm_detect.html. — 2026. — accessed Feb. 2026.
- [18] *YosysHQ.* FSM.\$fsm cell. — https://yosyshq.readthedocs.io/projects/yosys/en/latest/cell/word_fsm.html. — 2026. — accessed Feb. 2026.
- [19] Top level module for the UCE. — https://github.com/black-parrot/black-parrot/blob/master/bp_me/src/v/cce/bp_uce.sv. — 2026. — accessed Feb. 2026.

- [20] BlackParrot – A Linux-Capable Accelerator Host RISC-V Multicore. — <https://github.com/black-parrot/black-parrot>. — 2026. — accessed Feb. 2026.
- [21] CVA6 RISC-V CPU. — <https://github.com/openhwgroup/cva6>. — 2026. — accessed Feb. 2026.
- [22] Ibex RISC-V Core. — <https://github.com/lowRISC/ibex>. — 2026. — accessed Feb. 2026.
- [23] OpenXuantie – OpenC910 Core. — <https://github.com/XUANTIE-RV/openc910>. — 2026. — accessed Feb. 2026.
- [24] PicoRV32 – A Size-Optimized RISC-V CPU. — <https://github.com/YosysHQ/picorv32>. — 2026. — accessed Feb. 2026.
- [25] SCR1 RISC-V Core. — <https://github.com/syntacore/scr1>. — 2026. — accessed Feb. 2026.
- [26] VeeR EH1 RISC-V Core. — <https://github.com/chipsalliance/Cores-VeeR-EH1>. — 2026. — accessed Feb. 2026.
- [27] VeeR EL2 RISC-V Core. — <https://github.com/chipsalliance/Cores-VeeR-EL2>. — 2026. — accessed Feb. 2026.
- [28] Verilog implementation of the symmetric block cipher AES. — <https://github.com/secworks/aes>. — 2026. — accessed Feb. 2026.
- [29] OpenTitan: Open source silicon root of trust. — <https://github.com/lowRISC/opentitan>. — 2026. — accessed Feb. 2026.
- [30] Deadcode in FSM. — <https://github.com/XUANTIE-RV/openc910/issues/49>. — 2026. — accessed Feb. 2026.